

GUÍA DE EJERCICIOS RESUELTOS JAVA Y POO DESDE CERO

Documento de apoyo basado en el archivo de enunciados proporcionado

Materia/tema	Java y Programación Orientada a Objetos
Alumno	Rafael Morales
Contenido	Enunciados organizados + solución guía por tema
Fecha	Mayo 2026

Nota: Los códigos se presentan como soluciones guía. En clases reales de Java, cuando existan varias clases public, deben guardarse en archivos separados o dejar solo una clase public por archivo.

Índice

1. 1. Primeros pasos
2. 2. Variables y constantes
3. 3. Operadores
4. 4. Strings
5. 5. Condicionales
6. 6. Estructuras
7. 7. Bucles
8. 8. Funciones
9. 9. POO: Clases y objetos
10. 10. POO: Modificadores de acceso
11. 11. POO: Herencia
12. 12. POO: Polimorfismo
13. 13. POO: Abstracción
14. 14. Excepciones
15. 15. Extras

Introducción

El presente documento organiza los ejercicios del curso de Java y Programación Orientada a Objetos desde cero. Para cada tema se incluyen los enunciados originales y una solución guía en código Java. El objetivo es que el material sirva como apoyo para practicar sintaxis, estructuras básicas, funciones, clases, objetos, encapsulamiento, herencia, polimorfismo, abstracción y manejo de excepciones.

Recomendación de uso: crear un proyecto en un IDE como IntelliJ IDEA, Eclipse, NetBeans o Visual Studio Code, y probar cada bloque de código en un archivo independiente. Cuando el ejercicio incluya varias clases, se puede dejar una sola clase public y el resto como clases sin public dentro del mismo archivo.

1. Primeros pasos

Enunciados del ejercicio

16. 1. Imprime un mensaje que diga tu nombre en lugar de "¡Hola Mundo!".
17. 2. Imprime dos líneas: "Hola" y luego "Mundo" con un solo println.
18. 3. Añade un comentario sobre lo que hace cada línea del programa.
19. 4. Crea un comentario en varias líneas.
20. 5. Imprime tu edad, tu color favorito y tu ciudad.
21. 6. Explora los diferentes System.XXX.println(); más allá de "out".
22. 7. Utiliza varios println para imprimir una frase.
23. 8. Imprime un diseño ASCII (por ejemplo, una cara feliz usando símbolos).
24. 9. Intenta ejecutar el programa sin el método main y observa el error.
25. 10. Intenta cambiar el nombre del archivo a uno diferente del de la clase y compíllalo. ¿Qué pasa?

Solución guía en Java

```
public class PrimerosPasos {
    public static void main(String[] args) {
        // 1. Imprime un nombre
        System.out.println("Rafael");

        // 2. Dos líneas con un solo println
        System.out.println("Hola\nMundo");

        // 3. Comentarios por línea
        System.out.println("Esta línea imprime texto en consola"); // imprime un mensaje

        /* 4. Comentario de varias líneas:
           Este programa muestra ejemplos básicos de salida en Java. */

        // 5. Datos personales
        System.out.println("Edad: 21");
        System.out.println("Color favorito: azul");
        System.out.println("Ciudad: CDMX");

        // 6. Otros flujos de salida
        System.out.println("Mensaje normal");
        System.err.println("Mensaje de error");

        // 7. Frase con varios println
        System.out.println("Estoy aprendiendo");
        System.out.println("Java");
        System.out.println("desde cero");

        // 8. Diseño ASCII
        System.out.println("  :- )  ");
        System.out.println(" /|  |\\ ");
        System.out.println(" /  \\ ");

        // 9. Sin main: Java no sabrá desde dónde iniciar el programa.
        // 10. Si la clase es public, el archivo debe llamarse igual que la clase.
    }
}
```

Comentario: Este bloque concentra los ejercicios principales del tema en una sola clase para facilitar la ejecución y revisión. Puede modificarse con datos personales o valores diferentes.

2. Variables y constantes

Enunciados del ejercicio

26. 1. Declara una variable de tipo String y asígnale tu nombre.
27. 2. Crea una variable de tipo int y asígnale tu edad.
28. 3. Crea una variable double con tu altura en metros.
29. 4. Declara una variable de tipo boolean que indique si te gusta programar.
30. 5. Declara una constante con tu email.
31. 6. Crea una variable de tipo char y guárdale tu inicial.
32. 7. Declara una variable de tipo String con tu localidad, y a continuación cambia su valor y vuelve a imprimirla.
33. 8. Crea una variable int llamada a, otra b, e imprime la suma de ambas.
34. 9. Imprime el tipo de dos variables creadas anteriormente.
35. 10. Intenta declarar una variable sin inicializarla y luego asígnale un valor antes de imprimirla.

Solución guía en Java

```
public class VariablesConstantes {  
    public static void main(String[] args) {  
        String nombre = "Rafael";  
        int edad = 21;  
        double altura = 1.75;  
        boolean meGustaProgramar = true;  
        final String EMAIL = "correo@ejemplo.com";  
        char inicial = 'R';  
  
        String localidad = "CDMX";  
        System.out.println(localidad);  
        localidad = "Coapa";  
        System.out.println(localidad);  
  
        int a = 10;  
        int b = 15;  
        System.out.println("Suma: " + (a + b));  
  
        System.out.println("Tipo de nombre: String");  
        System.out.println("Tipo de edad: int");  
  
        String mensaje;  
        mensaje = "Variable inicializada después";  
        System.out.println(mensaje);  
    }  
}
```

Comentario: Este bloque concentra los ejercicios principales del tema en una sola clase para facilitar la ejecución y revisión. Puede modificarse con datos personales o valores diferentes.

3. Operadores

Enunciados del ejercicio

36. 1. Crea una variable con el resultado de cada operación aritmética.
37. 2. Crea una variable para cada tipo de operación de asignación.
38. 3. Imprime 3 comparaciones verdaderas con diferentes operadores de comparación.
39. 4. Imprime 3 comparaciones falsas con diferentes operadores de comparación.
40. 5. Utiliza el operador lógico and.
41. 6. Utiliza el operador lógico or.
42. 7. Combina ambos operadores lógicos.
43. 8. Añade alguna negación.
44. 9. Imprime 3 ejemplos de uso de operadores unarios.
45. 10. Combina operadores aritméticos, de comparación y lógicos.

Solución guía en Java

```
public class Operadores {  
    public static void main(String[] args) {  
        int x = 20, y = 6;  
        int suma = x + y;  
        int resta = x - y;  
        int multiplicacion = x * y;  
        int division = x / y;  
        int modulo = x % y;  
  
        int n = 10;  
        n += 5; n -= 2; n *= 3; n /= 2; n %= 4;  
  
        System.out.println(10 > 5);  
        System.out.println(10 >= 10);  
        System.out.println(5 != 3);  
  
        System.out.println(4 > 9);  
        System.out.println(8 == 2);  
        System.out.println(7 < 1);  
  
        boolean and = (10 > 5) && (3 < 8);  
        boolean or = (10 < 5) || (3 < 8);  
        boolean combinado = (x > y && y > 0) || x == 20;  
        boolean negacion = !(x < y);  
  
        int preIncremento = ++x;  
        int postIncremento = y++;  
        int negativo = -x;  
  
        boolean resultadoFinal = ((x + y) > 20) && (x != y);  
        System.out.println(resultadoFinal);  
    }  
}
```

Comentario: Este bloque concentra los ejercicios principales del tema en una sola clase para facilitar la ejecución y revisión. Puede modificarse con datos personales o valores diferentes.

4. Strings

Enunciados del ejercicio

46. 1. Concatena dos cadenas de texto.
47. 2. Muestra la longitud de una cadena de texto.
48. 3. Muestra el primer y último carácter de un string.
49. 4. Convierte a mayúsculas y minúsculas un string.
50. 5. Comprueba si una cadena de texto contiene una palabra concreta.
51. 6. Formatea un string con un entero.
52. 7. Elimina los espacios en blanco al principio y final de un string.
53. 8. Sustituye todos los espacios en blanco de un string por un guión (-).
54. 9. Comprueba si dos strings son iguales.
55. 10. Comprueba si dos strings tienen la misma longitud.

Solución guía en Java

```
public class StringsEjercicios {
    public static void main(String[] args) {
        String nombre = "Rafael";
        String apellido = "Morales";
        String completo = nombre + " " + apellido;

        System.out.println(completo.length());
        System.out.println(completo.charAt(0));
        System.out.println(completo.charAt(completo.length() - 1));
        System.out.println(completo.toUpperCase());
        System.out.println(completo.toLowerCase());
        System.out.println(completo.contains("Rafael"));

        int edad = 21;
        String texto = String.format("Tengo %d años", edad);
        System.out.println(texto);

        String frase = "    Java es muy útil    ";
        System.out.println(frase.trim());
        System.out.println(frase.trim().replace(" ", "-"));

        String a = "Hola";
        String b = "Hola";
        System.out.println(a.equals(b));
        System.out.println(a.length() == b.length());
    }
}
```

Comentario: Este bloque concentra los ejercicios principales del tema en una sola clase para facilitar la ejecución y revisión. Puede modificarse con datos personales o valores diferentes.

5. Condicionales

Enunciados del ejercicio

56. 1. Establece la edad de un usuario y muestra si puede votar (mayor o igual a 18).
57. 2. Declara dos números y muestra cuál es mayor, o si son iguales.
58. 3. Dado un número, verifica si es positivo, negativo o cero.
59. 4. Crea un programa que diga si un número es par o impar.
60. 5. Verifica si un número está en el rango de 1 a 100.
61. 6. Declara una variable con el día de la semana (1-7) y muestra su nombre con switch.
62. 7. Simula un sistema de notas: muestra "Sobresaliente", "Aprobado" o "Suspenso" según la nota (0-100).
63. 8. Escribe un programa que determine si puedes entrar al cine: debes tener al menos 15 años o ir acompañado.
64. 9. Crea un programa que diga si una letra es vocal o consonante.
65. 10. Usa tres variables a, b, c y muestra cuál es el mayor de las tres.

Solución guía en Java

```
public class Condicionales {
    public static void main(String[] args) {
        int edad = 18;
        System.out.println(edad >= 18 ? "Puede votar" : "No puede votar");

        int a = 20, b = 30;
        if (a > b) System.out.println("a es mayor");
        else if (b > a) System.out.println("b es mayor");
        else System.out.println("Son iguales");

        int numero = -5;
        if (numero > 0) System.out.println("Positivo");
        else if (numero < 0) System.out.println("Negativo");
        else System.out.println("Cero");

        System.out.println(numero % 2 == 0 ? "Par" : "Impar");
        System.out.println(numero >= 1 && numero <= 100 ? "En rango" : "Fuera de rango");

        int dia = 3;
        switch (dia) {
            case 1 -> System.out.println("Lunes");
            case 2 -> System.out.println("Martes");
            case 3 -> System.out.println("Miércoles");
            case 4 -> System.out.println("Jueves");
            case 5 -> System.out.println("Viernes");
            case 6 -> System.out.println("Sábado");
            case 7 -> System.out.println("Domingo");
            default -> System.out.println("Día inválido");
        }

        int nota = 85;
        if (nota >= 90) System.out.println("Sobresaliente");
        else if (nota >= 60) System.out.println("Aprobado");
        else System.out.println("Suspenso");

        boolean acompañado = false;
        System.out.println(edad >= 15 || acompañado ? "Puede entrar" : "No puede entrar");

        char letra = 'a';
        if ("aeiouAEIOU".indexOf(letra) >= 0) System.out.println("Vocal");
        else System.out.println("Consonante");
    }
}
```



```
        int c = 25;
        int mayor = Math.max(a, Math.max(b, c));
        System.out.println("Mayor: " + mayor);
    }
}
```

Comentario: Este bloque concentra los ejercicios principales del tema en una sola clase para facilitar la ejecución y revisión. Puede modificarse con datos personales o valores diferentes.

6. Estructuras

Enunciados del ejercicio

67. 1. Crea un Array con 5 valores e imprime su longitud.
68. 2. Modifica uno de los valores del Array e imprime el valor del índice antes y después de modificarlo.
69. 3. Crea un ArrayList vacío.
70. 4. Añade 4 valores al ArrayList y elimina uno a continuación.
71. 5. Crea un HashSet con 2 valores diferentes.
72. 6. Añade un nuevo valor repetido y otro sin repetir al HashSet.
73. 7. Elimina uno de los elementos del HashSet.
74. 8. Crea un HashMap donde la clave sea un nombre y el valor el número de teléfono. Añade tres contactos.
75. 9. Modifica uno de los contactos y elimina otro.
76. 10. Dado un Array, transfórmalo en un ArrayList, a continuación en un HashSet y finalmente en un HashMap con clave y valor iguales.

Solución guía en Java

```
import java.util.*;

public class Estructuras {
    public static void main(String[] args) {
        int[] numeros = {1, 2, 3, 4, 5};
        System.out.println("Longitud: " + numeros.length);
        System.out.println("Antes: " + numeros[2]);
        numeros[2] = 99;
        System.out.println("Después: " + numeros[2]);

        ArrayList<String> lista = new ArrayList<>();
        lista.add("Uno"); lista.add("Dos"); lista.add("Tres"); lista.add("Cuatro");
        lista.remove("Dos");

        HashSet<String> conjunto = new HashSet<>();
        conjunto.add("Java"); conjunto.add("P00");
        conjunto.add("Java"); conjunto.add("Backend");
        conjunto.remove("P00");

        HashMap<String, String> contactos = new HashMap<>();
        contactos.put("Ana", "5551112222");
        contactos.put("Luis", "5553334444");
        contactos.put("Rafa", "5557778888");
        contactos.put("Ana", "5559990000");
        contactos.remove("Luis");

        String[] arreglo = {"A", "B", "C"};
        ArrayList<String> arrayList = new ArrayList<>(Arrays.asList(arreglo));
        HashSet<String> hashSet = new HashSet<>(arrayList);
        HashMap<String, String> mapa = new HashMap<>();
        for (String valor : hashSet) mapa.put(valor, valor);
        System.out.println(mapa);
    }
}
```

Comentario: Este bloque concentra los ejercicios principales del tema en una sola clase para facilitar la ejecución y revisión. Puede modificarse con datos personales o valores diferentes.

7. Bucles

Enunciados del ejercicio

77. 1. Imprime los números del 1 al 10 usando while.
78. 2. Usa do-while para mostrar todos los valores de un ArrayList.
79. 3. Imprime los múltiplos de 5 del 1 al 50 usando for.
80. 4. Recorre un Array de 5 números e imprime la suma total.
81. 5. Usa un for para recorrer un Array y mostrar sus valores.
82. 6. Usa for-each para recorrer un HashSet y un HashMap.
83. 7. Imprime los números del 10 al 1 (descendiente) con un bucle for.
84. 8. Usa continue para saltar los múltiplos de 3 del 1 al
85. 20.
86. 9. Usa break para detener un bucle cuando encuentres un número negativo en un array.
87. 10. Crea un programa que calcule el factorial de un número dado.

Solución guía en Java

```
import java.util.*;

public class Bucles {
    public static void main(String[] args) {
        int i = 1;
        while (i <= 10) System.out.println(i++);

        ArrayList<String> nombres = new ArrayList<>(Arrays.asList("Ana", "Luis", "Rafa"));
        int j = 0;
        do {
            System.out.println(nombres.get(j));
            j++;
        } while (j < nombres.size());

        for (int n = 1; n <= 50; n++) if (n % 5 == 0) System.out.println(n);

        int[] nums = {2, 4, 6, 8, 10};
        int suma = 0;
        for (int n : nums) suma += n;
        System.out.println("Suma: " + suma);

        for (int k = 0; k < nums.length; k++) System.out.println(nums[k]);

        HashSet<String> set = new HashSet<>(Arrays.asList("A", "B"));
        for (String valor : set) System.out.println(valor);

        HashMap<String, Integer> map = new HashMap<>();
        map.put("Rafa", 21); map.put("Ana", 22);
        for (Map.Entry<String, Integer> e : map.entrySet()) System.out.println(e.getKey() + ": " +
e.getValue());

        for (int n = 10; n >= 1; n--) System.out.println(n);

        for (int n = 1; n <= 20; n++) {
            if (n % 3 == 0) continue;
            System.out.println(n);
        }

        int[] datos = {5, 8, 2, -1, 7};
        for (int n : datos) {
            if (n < 0) break;
            System.out.println(n);
        }
    }
}
```

```
    }  
  
    int factorial = 1;  
    int numero = 5;  
    for (int n = 1; n <= numero; n++) factorial *= n;  
    System.out.println("Factorial: " + factorial);  
    }  
}
```

Comentario: Este bloque concentra los ejercicios principales del tema en una sola clase para facilitar la ejecución y revisión. Puede modificarse con datos personales o valores diferentes.

8. Funciones

Enunciados del ejercicio

88. 1. Crea una función que imprima "¡Te doy la bienvenida al curso de Java desde cero!".
89. 2. Escribe una función que reciba un nombre como parámetro y salude a esa persona.
90. 3. Haz un método que reciba dos números enteros y devuelva su resta.
91. 4. Crea un método que calcule el cuadrado de un número ($n * n$).
92. 5. Escribe una función que reciba un número y diga si es par o impar.
93. 6. Crea un método que reciba una edad y retorne true si es mayor de edad (y false en caso contrario).
94. 7. Implementa una función que reciba una cadena y retorne su longitud.
95. 8. Crea un método que reciba un array de enteros, calcula su media y lo retorna.
96. 9. Escribe un método que reciba un número y retorna su factorial.
97. 10. Crea una función que reciba un `ArrayList<String>` y lo recorra mostrando cada elemento.

Solución guía en Java

```
import java.util.*;

public class Funciones {
    public static void bienvenida() { System.out.println("¡Te doy la bienvenida al curso de Java desde cero!"); }
    public static void saludar(String nombre) { System.out.println("Hola, " + nombre); }
    public static int restar(int a, int b) { return a - b; }
    public static int cuadrado(int n) { return n * n; }
    public static void parImpar(int n) { System.out.println(n % 2 == 0 ? "Par" : "Impar"); }
    public static boolean esMayorDeEdad(int edad) { return edad >= 18; }
    public static int longitud(String texto) { return texto.length(); }
    public static double media(int[] nums) { int suma = 0; for (int n : nums) suma += n; return (double) suma / nums.length; }
    public static int factorial(int n) { int r = 1; for (int i = 1; i <= n; i++) r *= i; return r; }
}

public static void mostrar(ArrayList<String> lista) { for (String e : lista)
System.out.println(e); }

public static void main(String[] args) {
    bienvenida();
    saludar("Rafael");
    System.out.println(restar(10, 4));
    System.out.println(cuadrado(5));
    parImpar(7);
    System.out.println(esMayorDeEdad(21));
    System.out.println(longitud("Java"));
    System.out.println(media(new int[]{8, 9, 10}));
    System.out.println(factorial(5));
    mostrar(new ArrayList<>(Arrays.asList("Java", "P00", "Funciones")));
}
```

Comentario: Este bloque concentra los ejercicios principales del tema en una sola clase para facilitar la ejecución y revisión. Puede modificarse con datos personales o valores diferentes.

9. POO: Clases y objetos

Enunciados del ejercicio

98. 1. Crea una clase Book con atributos title y author. Crea un objeto y muestra sus datos.
99. 2. Crea una clase Dog con un método bark() que imprima su sonido.
100. 3. Añade un constructor a la clase Book que reciba title y author.
101. 4. Crea una clase Car con atributos brand y model y un método showData().
102. 5. Crea una clase Student con atributo score y un método que diga si aprobó (mayor o igual a 60).
103. 6. Crea una clase BankAccount con atributo balance y un método deposit() que sume el saldo.
104. 7. Crea una clase Rectangle con métodos para calcular el área y el perímetro.
105. 8. Crea una clase Worker que reciba nombre y salario, y un método para mostrar su salario.
106. 9. Crea varios objetos Person y guárdalos en un ArrayList.
107. 10. Crea una clase Product y un método que aplique un descuento sobre su precio.

Solución guía en Java

```
import java.util.*;

class Book {
    String title, author;
    Book(String title, String author) { this.title = title; this.author = author; }
    void showData() { System.out.println(title + " - " + author); }
}
class Dog { void bark() { System.out.println("Guau"); } }
class Car { String brand, model; void showData() { System.out.println(brand + " " + model); } }
class Student { int score; boolean passed() { return score >= 60; } }
class BankAccount { double balance; void deposit(double amount) { balance += amount; } }
class Rectangle { double width, height; double area() { return width * height; } double
perimeter() { return 2 * (width + height); } }
class Worker { String name; double salary; Worker(String n, double s) { name = n; salary = s; }
void showSalary() { System.out.println(salary); } }
class Person { String name; Person(String n) { name = n; } }
class Product { double price; double applyDiscount(double percent) { return price - (price *
percent / 100); } }

public class ClasesObjetos {
    public static void main(String[] args) {
        Book book = new Book("Java desde cero", "Autor ejemplo");
        book.showData();
        new Dog().bark();
        Car car = new Car(); car.brand = "Audi"; car.model = "A4"; car.showData();
        Student st = new Student(); st.score = 85; System.out.println(st.passed());
        BankAccount acc = new BankAccount(); acc.deposit(500); System.out.println(acc.balance);
        Rectangle r = new Rectangle(); r.width = 5; r.height = 3; System.out.println(r.area());
        Worker w = new Worker("Rafa", 15000); w.showSalary();
        ArrayList<Person> persons = new ArrayList<>(); persons.add(new Person("Ana"));
        persons.add(new Person("Luis"));
        Product p = new Product(); p.price = 1000; System.out.println(p.applyDiscount(15));
    }
}
```

Comentario: En POO es común separar cada clase en su propio archivo. Para una práctica rápida, varias clases sin el modificador public pueden colocarse en el mismo archivo junto con una clase principal public.

10. POO: Modificadores de acceso

Enunciados del ejercicio

108. 1. Crea una clase Person con atributos privados name y age. Usa los métodos getName(), setName(), getAge() y setAge() para asignar y mostrar valores desde otra clase.
109. 2. Crea una clase Product con el atributo privado price. Añade el método setPrice(double price) que solo permita precios mayores a
110. 0.
111. 3. Crea una clase BankAccount con el atributo privado balance. Implementa los métodos deposit(double amount) y withdraw(double amount) que validen las cantidades correctamente.
112. 4. Crea una clase Book con el atributo privado title. Permite leerlo con el método getTitle() pero no modificarlo (sin setTitle()). El título debe asignarse solo por el constructor.
113. 5. Crea una clase Temperature con el atributo privado celsius. El método setCelsius(double celsius) solo debe aceptar valores entre -100 y
114. 100.
115. 6. Crea una clase User con los atributos privados username y password. Implementa los métodos setUsername(String username), setPassword(String password) y checkPassword(String inputPassword) que compare contraseñas.
116. 7. Crea una clase Employee con el atributo privado salary. Agrega el método raiseSalary(double percent) que solo permita aumentos positivos.
117. 8. Crea una clase Rectangle con los atributos privados width y height. Agrega setters y el método calculateArea() que devuelva el resultado de width * height.
118. 9. Crea una clase Student con el atributo privado grade. Agrega los métodos setGrade(int grade) y isPassed() que retorne true si la nota es mayor o igual a
119. 60.
120. 10. Crea una clase Car con el atributo privado speed. Agrega los métodos accelerate(int amount) que aumente la velocidad (máximo 120) y brake(int amount) que reduzca la velocidad (mínimo 0).

Solución guía en Java

```
class PersonAccess {
    private String name;
    private int age;
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getAge() { return age; }
    public void setAge(int age) { if (age >= 0) this.age = age; }
}
class ProductAccess {
    private double price;
    public void setPrice(double price) { if (price > 0) this.price = price; }
    public double getPrice() { return price; }
}
class BankAccountAccess {
    private double balance;
    public void deposit(double amount) { if (amount > 0) balance += amount; }
    public void withdraw(double amount) { if (amount > 0 && amount <= balance) balance -= amount; }
    public double getBalance() { return balance; }
}
class BookAccess {
    private String title;
    public BookAccess(String title) { this.title = title; }
    public String getTitle() { return title; }
}
class Temperature { private double celsius; public void setCelsius(double celsius) { if (celsius >= -100 && celsius <= 100) this.celsius = celsius; } }
```

```

class User { private String username, password; public void setUsername(String u){username=u;}
public void setPassword(String p){password=p;} public boolean checkPassword(String input){return
password.equals(input);} }
class Employee { private double salary; public void raiseSalary(double percent){ if(percent > 0)
salary += salary * percent / 100; } }
class RectangleAccess { private double width, height; public void setWidth(double w){ if(w>0)
width=w; } public void setHeight(double h){ if(h>0) height=h; } public double calculateArea()
{return width*height;} }
class StudentAccess { private int grade; public void setGrade(int grade){ if(grade>=0 &&
grade<=100) this.grade=grade;} public boolean isPassed(){return grade>=60;} }
class CarAccess { private int speed; public void accelerate(int amount){ speed = Math.min(120,
speed + amount); } public void brake(int amount){ speed = Math.max(0, speed - amount); } public
int getSpeed(){return speed;} }

public class ModificadoresAcceso {
    public static void main(String[] args) {
        PersonAccess p = new PersonAccess(); p.setName("Rafa"); p.setAge(21);
        System.out.println(p.getName() + " " + p.getAge());
    }
}

```

Comentario: En POO es común separar cada clase en su propio archivo. Para una práctica rápida, varias clases sin el modificador public pueden colocarse en el mismo archivo junto con una clase principal public.

11. POO: Herencia

Enunciados del ejercicio

121. 1. Crea una clase Vehicle con un método move(). Luego crea una subclase Car que herede de Vehicle y agrega el método honk().
122. 2. Define una clase Person con los atributos name y age. Luego crea una clase Student que agregue el atributo grade y un método study().
123. 3. Crea una clase Animal con el método makeSound(). Haz que Dog diga "Woof" y Cat diga "Meow" sobrescribiendo ese método.
124. 4. La clase Employee tiene los atributos name y salary. Manager hereda de Employee y agrega el atributo department.
125. 5. Crea una clase abstracta Shape con un método calculateArea(). Luego implementa ese método en Circle y Rectangle.
126. 6. Crea una clase Bird con el método fly(). Luego crea Eagle que sobrescriba fly() pero también llame al método original con super.fly().
127. 7. Haz una clase Device con un constructor que imprima "Device created". Luego crea Phone que herede de Device y en su constructor imprima "Phone ready".
128. 8. Account tiene un saldo y métodos para deposit() y withdraw(). SavingsAccount hereda y agrega un método addInterest().
129. 9. Crea una clase Vehicle y tres subclases: Car, Bike y Truck, cada una con un método describe() sobrescrito.
130. 10. Crea un ArrayList<Animal> que contenga instancias de Dog, Cat y Bird. Recorre la lista y llama a makeSound().

Solución guía en Java

```
import java.util.*;

class Vehicle { void move() { System.out.println("El vehículo se mueve"); } void describe()
{ System.out.println("Vehículo genérico"); } }
class Car extends Vehicle { void honk() { System.out.println("Beep beep"); } @Override void
describe(){ System.out.println("Soy un auto"); } }
class Bike extends Vehicle { @Override void describe(){ System.out.println("Soy una bicicleta"); }
}
class Truck extends Vehicle { @Override void describe(){ System.out.println("Soy un camión"); } }
class PersonBase { String name; int age; }
class StudentChild extends PersonBase { int grade; void study(){ System.out.println("Estudiando");
} }
class AnimalBase { void makeSound(){ System.out.println("Sonido animal"); } }
class DogChild extends AnimalBase { @Override void makeSound(){ System.out.println("Woof"); } }
class CatChild extends AnimalBase { @Override void makeSound(){ System.out.println("Meow"); } }
class BirdChild extends AnimalBase { @Override void makeSound(){ System.out.println("Tweet"); } }
class EmployeeBase { String name; double salary; }
class Manager extends EmployeeBase { String department; }
abstract class ShapeBase { abstract double calculateArea(); }
class CircleChild extends ShapeBase { double radius = 3; double calculateArea(){ return Math.PI *
radius * radius; } }
class RectangleChild extends ShapeBase { double w = 4, h = 5; double calculateArea(){ return w *
h; } }
class Bird { void fly(){ System.out.println("El ave vuela"); } }
class Eagle extends Bird { @Override void fly(){ super.fly(); System.out.println("El águila vuela
alto"); } }
class Device { Device(){ System.out.println("Device created"); } }
class Phone extends Device { Phone(){ System.out.println("Phone ready"); } }
class Account { double balance; void deposit(double a){balance+=a;} void withdraw(double a)
{if(a<=balance) balance-=a;} }
class SavingsAccount extends Account { void addInterest(){ balance += balance * 0.05; } }
```

```

public class Herencia {
    public static void main(String[] args) {
        Car car = new Car(); car.move(); car.honk();
        new StudentChild().study();
        new DogChild().makeSound(); new CatChild().makeSound();
        System.out.println(new CircleChild().calculateArea());
        new Eagle().fly(); new Phone();
        Vehicle[] vehicles = {new Car(), new Bike(), new Truck()};
        for (Vehicle v : vehicles) v.describe();
        ArrayList<AnimalBase> animals = new ArrayList<>(Arrays.asList(new DogChild(), new
CatChild(), new BirdChild()));
        for (AnimalBase a : animals) a.makeSound();
    }
}

```

Comentario: En POO es común separar cada clase en su propio archivo. Para una práctica rápida, varias clases sin el modificador public pueden colocarse en el mismo archivo junto con una clase principal public.

12. POO: Polimorfismo

Enunciados del ejercicio

131. 1. Crea una clase Animal con el método makeSound(). Luego crea subclases Dog, Cat y Cow que sobrescriban ese método con sonidos diferentes. Llama al método desde una lista de Animal.
132. 2. Crea una clase Shape con el método calculateArea(). Luego implementa subclases Circle y Rectangle con sus propias fórmulas. Usa una lista de Shape para recorrer e imprimir el área de varias figuras.
133. 3. Crea una clase Printer con varios métodos print() sobrecargados que acepten diferentes tipos de parámetros (String, int, double). Llama a cada uno desde main.
134. 4. Crea una clase Greeter con dos métodos greet(): uno que salude con "Hello", y otro que reciba un nombre y salude con "Hello, [nombre]".
135. 5. Crea una clase Vehicle con un método start(). Luego crea Car, Bike y Truck que sobrescriban ese método. Recorre una lista ArrayList<Vehicle> para llamar a start() en cada uno.
136. 6. Crea una clase Notification con método send(), y subclases EmailNotification, SMSNotification. Luego crea una función sendNotification(Notification n) que reciba cualquier tipo y lo ejecute.
137. 7. Crea una función showAnimalType(Animal animal) que imprima el tipo de animal. Pasa diferentes subclases (Dog, Cat, Horse) para que cada una imprima su tipo con su propio getType() sobrescrito.
138. 8. Crea una clase Converter con métodos convert(int), convert(double), y convert(String) que devuelvan diferentes formatos de texto.
139. 9. Crea una clase Product con el método getPrice(). Luego, Book y Electronic deben sobrescribirlo con su propia lógica de descuento. Recorre una lista de Product e imprime el precio final de cada uno.
140. 10. Crea una clase Character con método attack(). Luego crea subclases Warrior, Archer, Mage con ataques diferentes. En main, crea un array de Character y llama a attack() para cada uno.

Solución guía en Java

```
import java.util.*;

class AnimalPoly { void makeSound(){ System.out.println("Sonido"); } String getType(){ return "Animal"; } }
class DogPoly extends AnimalPoly { void makeSound(){System.out.println("Guau");} String getType(){return "Perro";} }
class CatPoly extends AnimalPoly { void makeSound(){System.out.println("Miau");} String getType(){return "Gato";} }
class CowPoly extends AnimalPoly { void makeSound(){System.out.println("Muuu");} }
class HorsePoly extends AnimalPoly { String getType(){return "Caballo";} }
abstract class ShapePoly { abstract double calculateArea(); }
class CirclePoly extends ShapePoly { double r=2; double calculateArea(){return Math.PI*r*r;} }
class RectanglePoly extends ShapePoly { double w=3,h=4; double calculateArea(){return w*h;} }
class Printer { void print(String s){System.out.println(s);} void print(int n){System.out.println(n);} void print(double d){System.out.println(d);} }
class Greeter { void greet(){System.out.println("Hello");} void greet(String name){System.out.println("Hello, " + name);} }
class VehiclePoly { void start(){System.out.println("Vehículo iniciado");} }
class CarPoly extends VehiclePoly { void start(){System.out.println("Auto iniciado");} }
class BikePoly extends VehiclePoly { void start(){System.out.println("Bici iniciada");} }
class TruckPoly extends VehiclePoly { void start(){System.out.println("Camión iniciado");} }
class Notification { void send(){System.out.println("Notificación");} }
class EmailNotification extends Notification { void send(){System.out.println("Email enviado");} }
class SMSNotification extends Notification { void send(){System.out.println("SMS enviado");} }
class Converter { String convert(int n){return "Entero: "+n;} String convert(double d){return "Decimal: "+d;} String convert(String s){return "Texto: "+s;} }
class ProductPoly { double getPrice(){return 0;} }
class BookPoly extends ProductPoly { double getPrice(){return 200 * 0.90;} }
class ElectronicPoly extends ProductPoly { double getPrice(){return 5000 * 0.85;} }
class CharacterGame { void attack(){System.out.println("Ataque genérico");} }
class Warrior extends CharacterGame { void attack(){System.out.println("Espadazo");} }
class Archer extends CharacterGame { void attack(){System.out.println("Flechazo");} }
```

```

class Mage extends CharacterGame { void attack(){System.out.println("Hechizo");} }

public class Polimorfismo {
    static void sendNotification(Notification n){ n.send(); }
    static void showAnimalType(AnimalPoly animal){ System.out.println(animal.getType()); }
    public static void main(String[] args) {
        ArrayList<AnimalPoly> animals = new ArrayList<>(Arrays.asList(new DogPoly(), new
CatPoly(), new CowPoly()));
        for (AnimalPoly a : animals) a.makeSound();
        ArrayList<ShapePoly> shapes = new ArrayList<>(Arrays.asList(new CirclePoly(), new
RectanglePoly()));
        for (ShapePoly s : shapes) System.out.println(s.calculateArea());
        Printer p = new Printer(); p.print("Hola"); p.print(10); p.print(2.5);
        new Greeter().greet("Rafa");
        ArrayList<VehiclePoly> vehicles = new ArrayList<>(Arrays.asList(new CarPoly(), new
BikePoly(), new TruckPoly()));
        for (VehiclePoly v : vehicles) v.start();
        sendNotification(new EmailNotification()); sendNotification(new SMSNotification());
        showAnimalType(new DogPoly()); showAnimalType(new CatPoly()); showAnimalType(new
HorsePoly());
        Converter c = new Converter(); System.out.println(c.convert(5));
        ArrayList<ProductPoly> products = new ArrayList<>(Arrays.asList(new BookPoly(), new
ElectronicPoly()));
        for(ProductPoly pr : products) System.out.println(pr.getPrice());
        CharacterGame[] characters = {new Warrior(), new Archer(), new Mage()};
        for (CharacterGame ch : characters) ch.attack();
    }
}

```

Comentario: En POO es común separar cada clase en su propio archivo. Para una práctica rápida, varias clases sin el modificador public pueden colocarse en el mismo archivo junto con una clase principal public.

13. POO: Abstracción

Enunciados del ejercicio

141. 1. Crea una clase abstracta Shape con el método calculateArea(). Luego implementa dos subclases: Circle y Rectangle, y haz que cada una calcule su propia área.
142. 2. Crea una interfaz Playable con el método play(). Luego implementa esa interfaz en dos clases: Guitar y Piano. Cada una debe mostrar un mensaje diferente al ejecutarse.
143. 3. Define una clase abstracta Animal con el método makeSound(). Implementa Dog y Cat para que hagan sonidos distintos. Crea un array de Animal para mostrar polimorfismo.
144. 4. Crea una interfaz Drawable. Implementa las clases Circle, Square, y Triangle que muestren cómo se dibuja cada figura usando draw().
145. 5. Crea una clase abstracta Employee con un método calculateSalary(). Implementa FullTimeEmployee y PartTimeEmployee con lógica diferente para calcular el salario.
146. 6. Crea una interfaz Movable con el método move(). Haz que las clases Car y Robot implementen ese método con comportamientos diferentes.
147. 7. Crea una clase abstracta Appliance con método turnOn() y turnOff. Implementa TV y WashingMachine con mensajes diferentes al encender y apagar.
148. 8. Crea dos interfaces Flyable y Swimmable. Crea una clase Duck que implemente ambas interfaces y muestre cómo puede volar y nadar.
149. 9. Crea una clase abstracta Document con el método print(). Luego crea PDFDocument y WordDocument, cada una con su forma de imprimir.
150. 10. Crea una interfaz Payable con el método pay(). Luego implementa las clases Invoice y EmployeePayment, cada una mostrando un mensaje de pago diferente.

Solución guía en Java

```
abstract class ShapeAbs { abstract double calculateArea(); }
class CircleAbs extends ShapeAbs { double r=3; double calculateArea(){return Math.PI*r*r;} }
class RectangleAbs extends ShapeAbs { double w=4,h=5; double calculateArea(){return w*h;} }
interface Playable { void play(); }
class Guitar implements Playable { public void play(){System.out.println("Tocando guitarra");} }
class Piano implements Playable { public void play(){System.out.println("Tocando piano");} }
abstract class AnimalAbs { abstract void makeSound(); }
class DogAbs extends AnimalAbs { void makeSound(){System.out.println("Guau");} }
class CatAbs extends AnimalAbs { void makeSound(){System.out.println("Miau");} }
interface Drawable { void draw(); }
class CircleDraw implements Drawable { public void draw(){System.out.println("Dibujando círculo");} }
class Square implements Drawable { public void draw(){System.out.println("Dibujando cuadrado");} }
class Triangle implements Drawable { public void draw(){System.out.println("Dibujando triángulo");} }
abstract class EmployeeAbs { abstract double calculateSalary(); }
class FullTimeEmployee extends EmployeeAbs { double calculateSalary(){return 15000;} }
class PartTimeEmployee extends EmployeeAbs { double calculateSalary(){return 80 * 100;} }
interface Movable { void move(); }
class CarMove implements Movable { public void move(){System.out.println("Auto en movimiento");} }
class Robot implements Movable { public void move(){System.out.println("Robot caminando");} }
abstract class Appliance { abstract void turnOn(); abstract void turnOff(); }
class TV extends Appliance { void turnOn(){System.out.println("TV encendida");} void turnOff(){System.out.println("TV apagada");} }
class WashingMachine extends Appliance { void turnOn(){System.out.println("Lavadora encendida");} void turnOff(){System.out.println("Lavadora apagada");} }
interface Flyable { void fly(); }
interface Swimmable { void swim(); }
class Duck implements Flyable, Swimmable { public void fly(){System.out.println("Pato vuela");} public void swim(){System.out.println("Pato nada");} }
abstract class DocumentAbs { abstract void print(); }
class PDFDocument extends DocumentAbs { void print(){System.out.println("Imprimiendo PDF");} }
```

```

class WordDocument extends DocumentAbs { void print(){System.out.println("Imprimiendo Word");} }
interface Payable { void pay(); }
class Invoice implements Payable { public void pay(){System.out.println("Factura pagada");} }
class EmployeePayment implements Payable { public void pay(){System.out.println("Empleado
pagado");} }

public class Abstraccion {
    public static void main(String[] args) {
        ShapeAbs[] shapes = {new CircleAbs(), new RectangleAbs()};
        for (ShapeAbs s : shapes) System.out.println(s.calculateArea());
        Playable[] instruments = {new Guitar(), new Piano()};
        for (Playable p : instruments) p.play();
        AnimalAbs[] animals = {new DogAbs(), new CatAbs()};
        for (AnimalAbs a : animals) a.makeSound();
        new Duck().fly(); new Duck().swim();
        new PDFDocument().print(); new WordDocument().print();
        new Invoice().pay(); new EmployeePayment().pay();
    }
}

```

Comentario: En POO es común separar cada clase en su propio archivo. Para una práctica rápida, varias clases sin el modificador public pueden colocarse en el mismo archivo junto con una clase principal public.

14. Excepciones

Enunciados del ejercicio

151. 1. Divide dos números almacenados en dos variables. Maneja la división por cero con try-catch.
152. 2. Crea un array de 3 elementos e intenta acceder al índice
153. 5. Captura el `ArrayIndexOutOfBoundsException`.
154. 3. Crea una variable `String` nula e intenta imprimir su longitud. Maneja el `NullPointerException`.
155. 4. Escribe una función que transforma texto a número. Usa try-catch para manejar entradas no válidas (`NumberFormatException`).
156. 5. Escribe un programa con un bloque `finally` que se ejecute siempre, haya o no error.
157. 6. Usa `throw` para lanzar un `IllegalArgumentException` si un número introducido negativo.
158. 7. Crea una clase `TemperatureChecker` que lanza una excepción personalizada si la temperatura es menor a -50 o mayor a 50.
159. 50.
160. 8. Crea un programa con varios bloques `catch`: uno para `ArithmeticException`, otro para `ArrayIndexOutOfBoundsException`.
161. 9. Crea una función `checkPassword(String pass)` que lance una excepción si la contraseña es demasiado corta.
162. 10. Implementa una clase `LoginSystem` que use una excepción personalizada `LoginFailedException` si el usuario o contraseña son incorrectos.

Solución guía en Java

```
class InvalidTemperatureException extends Exception {
    public InvalidTemperatureException(String message) { super(message); }
}
class LoginFailedException extends Exception {
    public LoginFailedException(String message) { super(message); }
}
class TemperatureChecker {
    void check(double temp) throws InvalidTemperatureException {
        if (temp < -50 || temp > 50) throw new InvalidTemperatureException("Temperatura fuera de rango");
    }
}
class LoginSystem {
    void login(String user, String pass) throws LoginFailedException {
        if (!user.equals("admin") || !pass.equals("1234")) throw new LoginFailedException("Usuario o contraseña incorrectos");
    }
}

public class Excepciones {
    static int textToNumber(String text) { return Integer.parseInt(text); }
    static void checkPassword(String pass) {
        if (pass.length() < 8) throw new IllegalArgumentException("Contraseña demasiado corta");
    }
    public static void main(String[] args) {
        try { int r = 10 / 0; } catch (ArithmeticException e) { System.out.println("No se puede dividir entre cero"); }
        try { int[] arr = {1,2,3}; System.out.println(arr[5]); } catch (ArrayIndexOutOfBoundsException e) { System.out.println("Índice inválido"); }
        try { String s = null; System.out.println(s.length()); } catch (NullPointerException e) { System.out.println("Texto nulo"); }
        try { System.out.println(textToNumber("abc")); } catch (NumberFormatException e) { System.out.println("No es número"); }
        try { System.out.println("Bloque try"); } finally { System.out.println("Finally siempre se ejecuta"); }
```

```

        int numero = -5;
        if (numero < 0) { /* throw new IllegalArgumentException("Número negativo"); */ }
        try { new TemperatureChecker().check(80); } catch (InvalidTemperatureException e)
{ System.out.println(e.getMessage()); }
        try { int a = 10 / 0; int[] b = {1}; System.out.println(b[2]); }
        catch (ArithmeticException e) { System.out.println("Error aritmético"); }
        catch (ArrayIndexOutOfBoundsException e) { System.out.println("Error de índice"); }
        try { checkPassword("123"); } catch (IllegalArgumentException e)
{ System.out.println(e.getMessage()); }
        try { new LoginSystem().login("admin", "x"); } catch (LoginFailedException e)
{ System.out.println(e.getMessage()); }
    }
}

```

Comentario: En POO es común separar cada clase en su propio archivo. Para una práctica rápida, varias clases sin el modificador public pueden colocarse en el mismo archivo junto con una clase principal public.

15. Extras

Enunciados del ejercicio

163. 1. Crea una variable de tipo String inicializada como null y verifica que no esté vacía antes de usarla.
164. 2. Escribe un programa que lea el nombre y edad del usuario usando Scanner.
165. 3. Declara una constante final llamada MAX_SCORE con valor 100 y muéstrala.
166. 4. Crea una variable global message y otra local message dentro del método main(). Muestra ambas.
167. 5. Usa import java.util.Scanner; para leer un número y mostrar si es positivo negativo.
168. 6. Declara una variable static en una clase y accede a ella desde main() sin crear un objeto.
169. 7. Importa java.util.Random y genera un número aleatorio del 1 al
170. 10.
171. 8. Crea una clase con comentarios adecuados explicando cada sección del código.
172. 9. Define una clase User con una constante APP_NAME, una variable global username y una función que imprima ambas.
173. 10. Haz debug del código implementado haciendo uso de sus diferentes herramientas.

Solución guía en Java

```
import java.util.*;

class GlobalExample {
    static String message = "Mensaje global";
}
class StaticExample {
    static int counter = 10;
}
class UserExtra {
    static final String APP_NAME = "Mi App";
    String username;
    void printData() { System.out.println(APP_NAME + " - " + username); }
}

public class Extras {
    static String message = "Mensaje global de la clase";
    public static void main(String[] args) {
        String text = null;
        if (text != null && !text.isEmpty()) System.out.println(text);
        else System.out.println("Texto nulo o vacío");

        Scanner scanner = new Scanner(System.in);
        // System.out.print("Nombre: "); String nombre = scanner.nextLine();
        // System.out.print("Edad: "); int edad = scanner.nextInt();

        final int MAX_SCORE = 100;
        System.out.println(MAX_SCORE);

        String message = "Mensaje local";
        System.out.println(message);
        System.out.println(Extras.message);

        // int number = scanner.nextInt();
        // System.out.println(number >= 0 ? "Positivo" : "Negativo");

        System.out.println(StaticExample.counter);

        Random random = new Random();
        int aleatorio = random.nextInt(10) + 1;
        System.out.println(aleatorio);
    }
}
```

```
UserExtra user = new UserExtra();
user.username = "Rafael";
user.printData();

    // Debug sugerido: colocar breakpoints, ejecutar paso a paso y revisar variables.
}
}
```

Comentario: En POO es común separar cada clase en su propio archivo. Para una práctica rápida, varias clases sin el modificador public pueden colocarse en el mismo archivo junto con una clase principal public.

Conclusión

Los ejercicios permiten avanzar gradualmente desde la impresión de mensajes y el uso de variables hasta conceptos más avanzados de programación orientada a objetos. Practicar cada bloque ayuda a comprender la lógica de Java, mejorar la lectura de código y fortalecer la capacidad para diseñar programas organizados, reutilizables y mantenibles.

Referencias

Archivo base proporcionado por el usuario: "Enunciados ejercicios curso de Java y POO desde cero".

Oracle. (s. f.). The Java™ Tutorials. Oracle Java Documentation.